Technische Hochschule Deggendorf

Case Study: Intelligent Systems in Production

Early Warning of Air-Leak Failures in Metro Compressors: Explainable Anomaly Detection on the MetroPT-3 Dataset

Methodology Report – Phase 1:
Theoretical Framework and System Design

Submitted by (Group 6):

Fuad Obiedat
Abdennour Djeha
Mohamed Fares
Hady Nasser

Course:

M.Eng. in Applied AI for Digital Production Management

Under the Supervision of:

Prof. Dr.-Ing. Hamidreza Heidari

Date of Submission: 24.11.2026

Abstract

This report establishes the theoretical and methodological foundations of our case study on early warning of air-leak failures in the Air Production Unit (APU) of a Metro do Porto train, using the public **MetroPT-3** dataset with four documented failures. We first set out the theoretical framework: predictive maintenance formulated as unsupervised anomaly detection, the physical signature of an air leak in a compressed-air system, the isolation and reconstruction principles behind the selected detectors, event-level evaluation, and Shapley-value explanations. We then define the methodology as a controlled benchmark in which a **threshold rule**, an **Isolation Forest** and an **LSTM autoencoder** share the same preprocessing, the same alarm logic and the same evaluation protocol, so that the detector is the only difference between them. The data pipeline removes frozen logging segments, aggregates the signals into gap-aware 10-minute windows, splits the data chronologically (training on February 2020 only) and fits all scaling on the training period. Alarms are raised with a threshold of $Q_3 + 3 \cdot \mathrm{IQR}$ on the training scores followed by a low-pass filter, and are evaluated per failure with lead time, false alarms per day and lead time relative to the low-pressure signal. We describe the modular system architecture, the inputs and outputs, and the main assumptions and challenges, including the small number of failures and a distribution shift between the training and test months. A first prototype of the preprocessing pipeline and the Isolation Forest detects all four failures, one of them two hours early, and reacts 10–20 minutes earlier than the threshold rule for the others, but raises 0.52 false alarms per day compared with 0.25, mainly because of the distribution shift.

Keywords: predictive maintenance; anomaly detection; Isolation Forest; LSTM autoencoder; event-level evaluation; SHAP; MetroPT-3

Contents

1 Introduction **4**
1.1 Objectives 4

2 Theoretical Framework **4**
2.1 Predictive Maintenance as Early Failure Warning 4
2.2 Physical Basis: Compressor Operation and Air Leaks 5
2.3 Isolation-Based Anomaly Detection 5
2.4 Reconstruction-Based Detection with LSTM Autoencoders 6
2.5 From Scores to Alarms 6
2.6 Event-Level Evaluation 7
2.7 Explainability with Shapley Values 7

3 Materials and Methods **7**
3.1 Research Approach 7
3.2 Workflow 8
3.3 Data and Inputs 8
3.4 Preprocessing 9
3.5 Algorithms and Reasons for Their Selection 10
3.6 Initial Design of the LSTM Autoencoder 11
3.7 Alarm Logic 12
3.8 Evaluation Protocol 12
3.9 Reproducibility 12

4 System Architecture **12**

5 Initial Prototype **13**

6 Assumptions and Challenges **16**
6.1 Assumptions 16
6.2 Limitations and Challenges 16

7 Conclusion **17**

List of Figures

1 Daily share of time under load (motor current > 4.7 A). Shaded: reported failures. Source: notebook `01_EDA`. 5
2 Workflow. Blue: shared by all detectors; orange: detector-specific; green: explanation (phase 2). 8
3 Signals around failure 1. All signals are flat from 17 April 09:20 to 18 April 00:18 (frozen logging). Source: notebook `01_EDA`. 10
4 System architecture. Each layer only uses the output of the layer above. 13

5 Isolation Forest scores and alarms on the test period. Red: reported failures; grey: removed frozen segments. 15
6 Absolute mean of the scaled test features (0 = same as February). The 15 largest shifts are shown. 16

List of Tables

1 MetroPT-3 signals (descriptions from the dataset page) and their use in this study. 8
2 Selected algorithms and reasons for selection. 11
3 Components, code modules and interfaces. 13
4 Prototype results on the test period (March–August 2020), horizon 2 h. IF = Isolation Forest. 14
5 Lead time per failure in minutes (positive = before the reported start) and first LPS activation within $[t_s - 2\,\mathrm{h},\, t_e]$. 14

1 Introduction

The Air Production Unit (APU) of a Metro do Porto train supplies compressed air to pneumatic consumers such as the secondary suspension. Because the unit has no redundancy, a failure removes the train from service, and the operator requires failures to be detected at least two hours before the train becomes non-operational, with few false alarms [1]. Our case study addresses this requirement on the public MetroPT-3 dataset [2], which contains 1,516,948 records of 15 APU signals from February to August 2020 and four air-leak failures documented by maintenance reports.

The previous milestones defined the project and reviewed the literature. The bibliography report concluded that the task is best framed as unsupervised early failure warning, that simple detectors are often competitive with deep models [3], that point-adjusted metrics can overstate performance [4], and that the closest prior study already applies an LSTM autoencoder with SHAP to MetroPT-3 [5]. Our contribution is therefore not a new model but a transparent benchmark: a threshold rule, an Isolation Forest and an LSTM autoencoder compared under one leakage-free protocol with event-level metrics, on data whose quality issues are handled explicitly. A first exploratory analysis found recording gaps covering 17.7 % of the period, nine frozen logging segments and a distribution shift between February and the later months.

1.1 Objectives

This report covers the first methodology phase. Its objectives are to

1. set out the theoretical framework that supports the proposed solution (Section 2);
2. define the research approach, workflow, data, algorithms and evaluation protocol, and justify the choice of each algorithm (Section 3);
3. present the initial system architecture and the interaction of its components (Section 4);
4. report a first prototype of the preprocessing pipeline and the Isolation Forest (Section 5); and
5. state the main assumptions, limitations and implementation challenges (Section 6).

The detailed development of the LSTM autoencoder, the explanation module and the full experiments belong to the second methodology phase.

2 Theoretical Framework

2.1 Predictive Maintenance as Early Failure Warning

Predictive maintenance (PdM) monitors the condition of an asset so that interventions are planned when degradation is observed, instead of after a breakdown (corrective maintenance) or at fixed intervals (preventive maintenance) [6]. Within PdM, *failure prediction* asks whether a failure will occur within a horizon, while *remaining useful life* estimation predicts the time to the end of life [6]. With only four failures, a degradation trajectory cannot be

learned, so we formulate the task as *early failure warning*: an alarm should be raised within a fixed horizon before a failure, and not otherwise.

Labelling continuous sensor streams is costly, which makes supervised learning infeasible in railway PdM and motivates treating it as anomaly detection on time series [6]. An anomaly is "an observation that deviates considerably from some concept of normality" [7]. In our setting, the concept of normality is learned from a period in which the APU is assumed to work correctly, and every later observation is scored by how far it deviates from that concept. This is a semi-supervised setting in the terminology of Ruff et al. [7]: the training data are assumed to be normal, but no failure labels are used for training.

2.2 Physical Basis: Compressor Operation and Air Leaks

According to the dataset description, the MPG signal starts the compressor *under load* when the pressure falls below 8.2 bar, so that it delivers air to the reservoirs; otherwise the compressor runs *offloaded* or is switched off, and the description gives motor currents of about 0, 4 and 7 A for these states. The low-pressure switch (LPS) is activated when the pressure falls below 7 bar [2]. In the data, the motor current shows three clusters at about 0 A (off), 3.8 A (offloaded) and 5.4–6.3 A (under load); we therefore classify a record as *under load* when the motor current exceeds 4.7 A.

An air leak continuously removes air from the system. To keep the pressure, the compressor must run under load more often and for longer. The share of time under load (the *duty cycle*) is therefore the most direct physical indicator of a leak. Figure 1 confirms this in MetroPT-3: averaged over the four failures, the duty cycle is 0.975 during a failure, compared with 0.145 in normal operation. In the 24 hours before a failure, however, it is only 0.161, close to normal. The signature of a leak is strong once the leak is severe and weak beforehand, which is exactly what makes early warning difficult and motivates multivariate detectors that can combine several weak indications (pressures, current, temperature, valve activity).

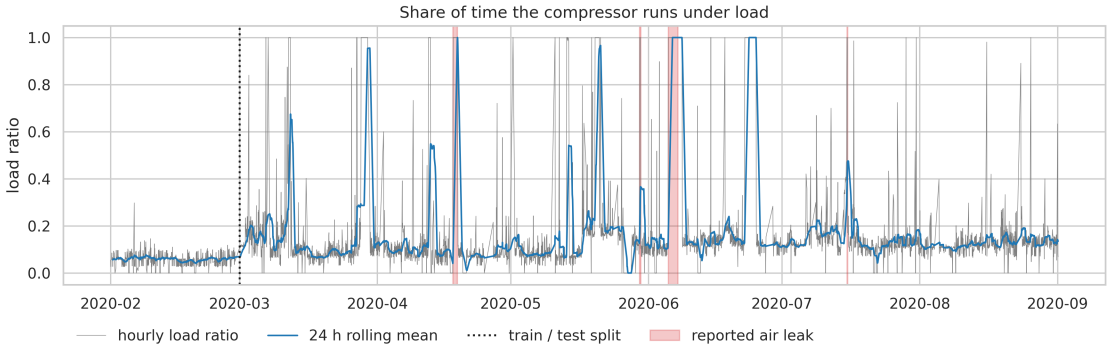


Figure 1: Daily share of time under load (motor current > 4.7 A). Shaded: reported failures. Source: notebook `01_EDA`.

2.3 Isolation-Based Anomaly Detection

Isolation Forest [8] does not model normal behaviour explicitly; it isolates individual points. Each tree is built on a random sub-sample by repeatedly choosing a random feature and

a random split value between its minimum and maximum, until every point is isolated. Anomalies are few and different, so they are separated after fewer splits and have a shorter path length $h(x)$ from the root. The anomaly score of a point x for a sub-sample size ψ is

$$s(x, \psi) = 2^{-\frac{\mathrm{E}[h(x)]}{c(\psi)}}, \qquad c(\psi) = 2H(\psi - 1) - \frac{2(\psi - 1)}{\psi}, \tag{1}$$

where $\mathrm{E}[h(x)]$ is the mean path length over all trees, $c(\psi)$ is the average path length of an unsuccessful search in a binary search tree and normalises the score, and $H(i) \approx \ln(i) + 0.5772$ is the harmonic number. Scores close to 1 indicate anomalies; scores well below 0.5 indicate normal points. The method has linear time complexity and low memory requirements and works when the training data contain no anomalies [8].

2.4 Reconstruction-Based Detection with LSTM Autoencoders

An autoencoder learns to compress its input into a low-dimensional representation and to reconstruct the input from it. Trained on normal data only, it reconstructs normal patterns well and unusual patterns poorly, so the reconstruction error serves as an anomaly score; autoencoders can capture nonlinear structure that linear methods such as PCA miss [9]. For sequences, the encoder and decoder are built from long short-term memory (LSTM) units [10], which keep information over many time steps. Malhotra et al. [11] train such an LSTM encoder–decoder on normal multi-sensor sequences and use the reconstruction error as the anomaly score. For a sequence $X = (x_1, \ldots, x_L)$ with $x_t \in \mathbb{R}^d$ and reconstruction $\hat{X}$, we use the mean squared error

$$e(X) = \frac{1}{L\,d} \sum_{t=1}^{L} \lVert x_t - \hat{x}_t \rVert_2^2. \tag{2}$$

LSTM autoencoders with a threshold on the reconstruction error are the most common detector in the Metro do Porto literature [5, 12, 13].

2.5 From Scores to Alarms

A continuous score must be turned into a binary decision. Following Silva et al. [12], the threshold is the upper outlier fence of the training scores,

$$\tau = Q_3 + k \cdot (Q_3 - Q_1), \qquad k = 3, \tag{3}$$

where Q_1 and Q_3 are the first and third quartiles. Isolated exceedances are often noise. Ribeiro et al. [14] raise an alarm only for sequences of abnormal observations by passing the binary exceedance signal $x_i \in \{0, 1\}$ through a low-pass filter,

$$y_i = y_{i-1} + \alpha\,(x_i - y_{i-1}), \tag{4}$$

and signalling an alarm when $y_i > 0.5$, which strongly reduced false alarms in their railway study. Smoothing errors before thresholding to reduce false positives is also used by Hundman et al. [15], whose operators noted that "investigation of even a couple false positives can deter users".

2.6 Event-Level Evaluation

Anomaly-detection benchmarks commonly report point-wise precision, recall and F1. For an early-warning system these measures are misleading for two reasons. First, the quantity of interest is whether, and how early, each *failure* is announced, not how many time steps are labelled correctly [16, 17]. Second, the widely used point adjustment, which counts a whole anomalous segment as detected if a single point in it is flagged, lets even random scores reach an F1 of 0.969 on a benchmark where the unadjusted F1 is 0.216 [4]. Benchmark datasets themselves can also be flawed [18]. We therefore evaluate per failure event and per alarm episode, without point adjustment (Section 3.8).

2.7 Explainability with Shapley Values

An alarm is useful only if maintenance staff understand why it was raised; lack of interpretability is a recognised source of mistrust in data-driven PdM [19]. SHAP [20] explains a single prediction $f(x)$ as an additive attribution

$$f(x) = \phi_0 + \sum_{j=1}^{M} \phi_j, \tag{5}$$

where ϕ_0 is the expected output over a background data set and ϕ_j is the contribution of feature j. The ϕ_j are Shapley values from cooperative game theory, the unique attribution that satisfies local accuracy, missingness and consistency [20]. Kernel SHAP estimates them for any model by evaluating it on perturbed inputs. SHAP is the most frequently used explanation method in PdM [21]. In our case, f is the anomaly score of a window and the background set is the training period, so ϕ_j states how much feature j pushes a window away from typical February behaviour.

3 Materials and Methods

3.1 Research Approach

The study is a quantitative, experimental comparison. Three detectors of increasing complexity are applied to the same data under the same protocol:

1. a **threshold rule** that encodes the physical leak signature directly (milestone 2);
2. an **Isolation Forest**, a fast, shallow multivariate detector; and
3. an **LSTM autoencoder**, a deep sequence model.

All three share the preprocessing and the evaluation; the two learned detectors also share the alarm logic. The detector is therefore the only independent variable, and differences in the results can be attributed to it. A second, smaller comparison (ablation) repeats the learned detectors without removing frozen segments, to measure the effect of the data-quality handling.

3.2 Workflow

Figure 2 shows the workflow. Steps 1–4 are common to all detectors, steps 5–6 are detector-specific, and steps 7–8 are again common.

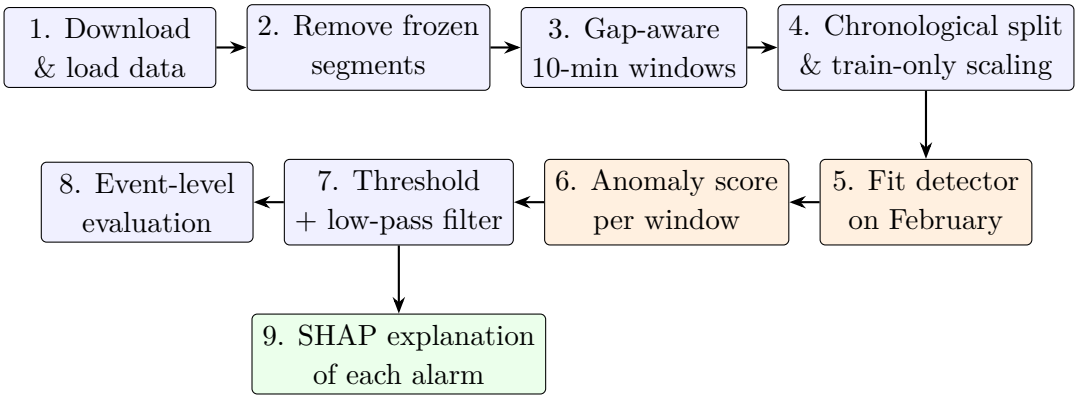


Figure 2: Workflow. Blue: shared by all detectors; orange: detector-specific; green: explanation (phase 2).

3.3 Data and Inputs

Source. MetroPT-3 [2] is downloaded by a script from the UCI Machine Learning Repository (CC BY 4.0) and is never stored in the repository. The four failure periods published on the dataset page are the only ground truth. Table 1 lists the signals and how each one is used.

Table 1: MetroPT-3 signals (descriptions from the dataset page) and their use in this study.

Signal	Description	Unit / type	Use
TP2	Pressure on the compressor	bar	feature
TP3	Pressure at the pneumatic panel	bar	feature
H1	Pressure drop at cyclonic separator filter	bar	feature
DV_pressure	Pressure drop at air-dryer tower discharge	bar	feature
Reservoirs	Downstream reservoir pressure	bar	feature
Oil_temperature	Compressor oil temperature	°C	feature
Motor_current	Current of one motor phase	A	feature, load state
COMP	Air intake valve	binary	feature
DV_electric	Compressor outlet valve	binary	feature
Towers	Active air-drying tower	binary	feature
MPG	Starts compressor under load (< 8.2 bar)	binary	feature
LPS	Low-pressure switch (< 7 bar)	binary	evaluation reference
Pressure_switch	Discharge in the air-drying towers	binary	feature
Oil_level	Oil below expected level	binary	feature
Caudal_impulses	Air-flow pulse counter	binary	feature

LPS is deliberately excluded from the inputs. It is a symptom of low pressure that already reflects a developing failure, and the operator-oriented criterion of Silva et al. [12] measures early warning relative to it; using it as a feature would make the evaluation circular.

Input variables. For each 10-minute window, the model input consists of the mean, standard deviation, minimum and maximum of the seven analogue signals (28 features), the share of active time of the seven digital signals other than LPS (7 features) and the share of time under load (1 feature), i.e. 36 features. The LSTM autoencoder will additionally use the scaled raw signals within each window as a sequence (Section 3.6).

Output variables. For every window the system produces (i) an anomaly score, (ii) a binary alarm and, for alarms, (iii) a ranked list of the features that contribute most to the score. For every failure, the evaluation produces a detection flag and the lead time.

Split. February 2020 (windows ending on or before 1 March 2020) is the training period; March to August 2020 is the test period, which contains all four failures. Taking the first month as normal training data and the following months as test data follows Silva et al. [12]. No information from the test period is used for training, scaling or setting thresholds.

3.4 Preprocessing

1. **Frozen segments.** Segments in which all seven analogue signals keep exactly the same value for at least 30 minutes are removed. Under operation at least one pressure or temperature changes within minutes, so such segments indicate a stuck logger. Nine segments (3.3 % of records) were found, one of them in the 15 hours before failure 1 (Figure 3).
2. **Gap-aware windows.** Records are aggregated into windows $[t - 10\,\text{min},\, t)$ indexed by the end time t, the earliest moment at which a decision can be made, so lead times are not inflated by the window length. A window is kept only if it contains at least 50 % of the expected 60 records; gaps are not filled with interpolated values.
3. **Chronological split** as described above.
4. **Scaling.** Each feature is standardised with the mean and standard deviation of the training windows only.

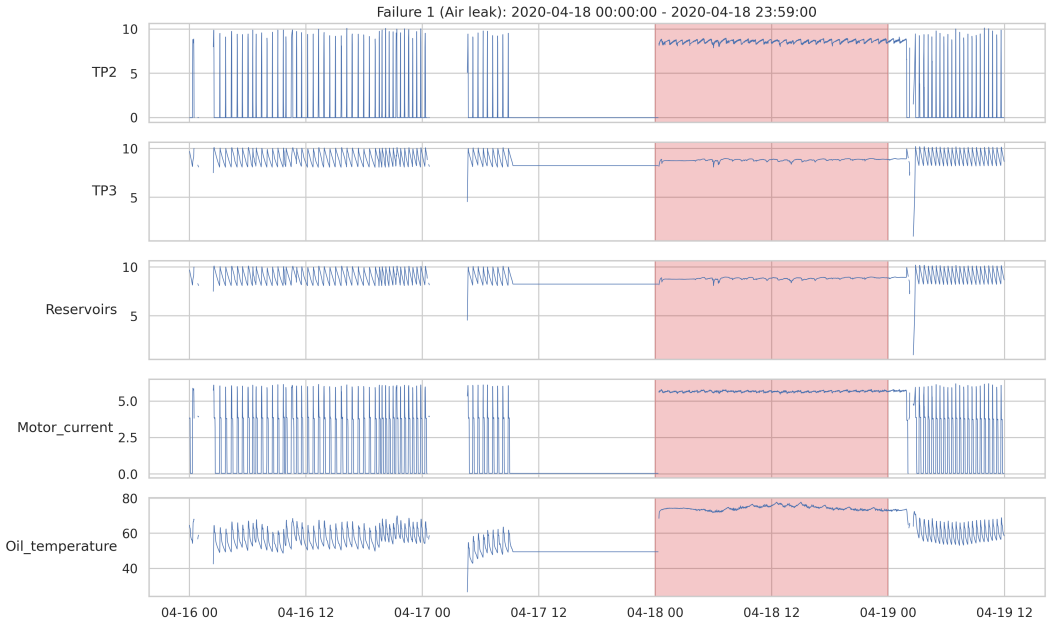


Figure 3: Signals around failure 1. All signals are flat from 17 April 09:20 to 18 April 00:18 (frozen logging). Source: notebook `01_EDA`.

3.5 Algorithms and Reasons for Their Selection

Table 2 summarises the selected algorithms and why each was chosen.

Table 2: Selected algorithms and reasons for selection.

Algorithm	Principle	Reasons for selection
Threshold rule	Training quantiles of four physically motivated window features; at least 2 of 4 rules for 3 consecutive windows	Directly encodes the leak signature (Section 2.2); fully interpretable; lower bound that learned models must beat.
Isolation Forest [8]	Anomalies are isolated by fewer random splits (Eq. 1)	Multivariate, trained on normal data only, fast and with few hyper-parameters; simple methods are often competitive with deep models [3]; represents the "simple" side of gap G2.
LSTM autoencoder [11]	Reconstruction error of normal sequences (Eq. 2)	Models temporal dependencies within a window; standard detector in the APU literature [12, 13], including on MetroPT-3 [5]; makes our results comparable.
Threshold $Q_3 + 3\,\mathrm{IQR}$ + low-pass filter	Eq. 3 and 4	Uses training scores only; suppresses isolated exceedances; same rule for both learned detectors, as used in [12, 14].
Kernel SHAP [20]	Shapley-value attribution of the anomaly score (Eq. 5)	Model-agnostic, so the same explanation method applies to both learned detectors; most used method in explainable PdM [21].

Alternatives were considered and not selected. Supervised classifiers would need labelled failures for training, and only four exist. Distance- and density-based detectors such as nearest-neighbour methods were not selected because Isolation Forest offers linear time complexity and low memory use, and compared favourably with such methods in its original evaluation [8]. Generative adversarial models performed best in Silva et al. [12], but are harder to train and tune; for a benchmark whose aim is a fair comparison of a simple and a deep model, the LSTM autoencoder is the established representative.

3.6 Initial Design of the LSTM Autoencoder

The LSTM autoencoder receives, for each 10-minute window, the sequence of the scaled raw analogue and digital signals (without LPS) at their native resolution of about 10 s, i.e. a sequence of up to 60 steps with 14 channels. Windows with less than 50 % coverage are skipped, as for the window features. The initial architecture is symmetric: an encoder with two LSTM layers (64 and 16 units) compresses the sequence into a 16-dimensional representation, which is repeated over the sequence length and decoded by two LSTM layers (16 and 64 units) and a linear output layer to the 14 channels. The model is trained with the mean squared error (Eq. 2) and the Adam optimiser; the last 20 % of February are held out chronologically for early stopping. The anomaly score of a window is its reconstruction error, which then passes through the same alarm logic as the Isolation Forest score. Layer sizes, sequence representation and training settings will be finalised in the second phase.

3.7 Alarm Logic

For the learned detectors, the threshold τ is computed from the training scores with Eq. 3 ($k = 3$). The exceedance signal $x_i = \mathbf{1}[s_i > \tau]$ is filtered with Eq. 4 using $\alpha = 0.3$; with this value, two consecutive exceeding windows are needed for a first alarm. The filter is reset to zero after a recording gap, so windows on either side of a gap are never treated as consecutive. The threshold rule keeps its own persistence filter (three consecutive windows) from milestone 2.

3.8 Evaluation Protocol

The evaluation is applied identically to all detectors on the test period with a warning horizon of $h = 2\,\mathrm{h}$, the operator requirement [1].

- **Detection.** A failure with reported start t_s and end t_e is detected if an alarm occurs in $[t_s - h,\, t_e]$.
- **Lead time.** $t_s - t_a$ in minutes, where t_a is the first alarm in that interval; positive values are early warnings, negative values are detection delays.
- **Lead time relative to LPS.** The same difference measured to the first LPS activation in $[t_s - h,\, t_e]$, corresponding to the criterion of Silva et al. [12].
- **False alarms.** Consecutive alarm windows form an episode; episodes that do not overlap any interval $[t_s - h,\, t_e]$ are false alarms, reported as a count and per day.
- **Window-level precision, recall and F1** without point adjustment, as a secondary view.

Because there are only four failures, results are reported per failure rather than only as averages.

3.9 Reproducibility

All code is written in Python with pinned package versions (`requirements.txt`) and a fixed random seed (42). The dataset is downloaded by `scripts/download_data.py`. All notebooks can be run on Google Colab, where the first cell clones the repository and downloads the data. Code and results are available at `https://github.com/mohamedfarez/IntelligentSystemsCaseStudy`.

4 System Architecture

The system is organised in five layers (Figure 4). Each layer is a Python module with a small interface, so detectors can be exchanged without changing the rest of the pipeline. Table 3 maps the components to the code.

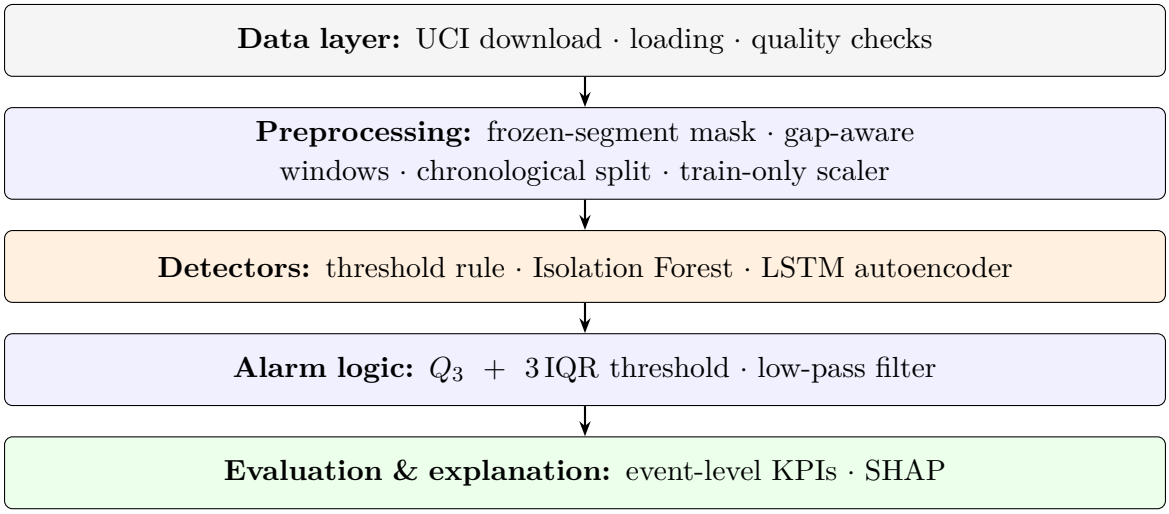


Figure 4: System architecture. Each layer only uses the output of the layer above.

Table 3: Components, code modules and interfaces.

Component	Module	Interface (input → output)
Configuration	`src/config.py`	Paths, schema, seed, failure periods, split date
Data loading	`src/data_loading.py`	CSV → time-indexed frame; quality report
Preprocessing	`src/preprocessing.py`	Raw frame → scaled train / test windows
Threshold rule	`src/threshold_baseline.py`	Window features → alarms
Learned detectors	`src/detectors.py`	`fit(train)`; `score(X)` → score per window
Alarm logic	`src/alarm_logic.py`	`fit(train scores)`; `predict(scores)` → alarms
Evaluation	`src/evaluation.py`	Alarms → per-failure table and KPIs
Experiment runner	`src/run_experiment.py`	Raw frame + detector name → all results
Explanation (phase 2)	`src/explain.py`	Detector + alarm windows → SHAP values

The components interact in one direction only. The preprocessing never sees detector outputs, the alarm logic is fitted on training scores only, and the evaluation receives nothing but the alarm series and the failure periods. This separation makes leakage from the test period structurally impossible and guarantees that all detectors are evaluated by the same code.

5 Initial Prototype

The preprocessing pipeline, the Isolation Forest, the common alarm logic and the shared evaluation are implemented (notebook `02_preprocessing_iforest`). The Isolation Forest uses 200 trees, a sub-sample size of 256 as recommended by Liu et al. [8] and the fixed seed. No parameter was tuned on the test period.

Verification. Before the run on the real data, the code was checked in three ways. (i) The evaluation functions were moved from the threshold baseline into a shared module; the baseline was re-run and produced identical results. (ii) The pipeline was run on a synthetic data set with the same schema, sampling interval and period as MetroPT-3, containing four simulated leaks, a recording gap and an inserted frozen segment before the first leak. The frozen segment was found and removed, and all four simulated leaks were detected. (iii) Without frozen-segment removal, the inserted frozen segment before the first simulated leak produced an apparent early warning of 120 minutes that disappeared once the segment was removed. On synthetic data this shows that frozen data can create alarms that look like early warnings but are caused by the logger, not by the machine.

Results on MetroPT-3. After frozen-segment removal (nine segments, 50,716 records, 3.3 %), 24,235 windows remain: 3,552 for training and 20,683 for testing. The threshold computed from the February scores is $\tau = 0.637$; 9.8 % of the test windows exceed it. Table 4 compares the Isolation Forest with the threshold rule of milestone 2 under the shared evaluation, and Table 5 gives the lead time per failure. The baseline values are identical to those reported in milestone 2.

Table 4: Prototype results on the test period (March–August 2020), horizon 2 h. IF = Isolation Forest.

Metric	Threshold rule	IF	IF, no frozen mask
Failures detected	4 / 4	4 / 4	4 / 4
Early warnings (before start)	1	1	1
Median lead time [min]	−20	−5	−5
False-alarm episodes	45	96	100
False alarms per day	0.245	0.522	0.543
Window precision	0.324	0.421	0.290
Window recall	0.911	0.947	0.921
Window F1	0.478	0.583	0.442

Table 5: Lead time per failure in minutes (positive = before the reported start) and first LPS activation within $[t_s - 2\,\text{h},\ t_e]$.

Failure	Threshold rule	IF	IF, no mask	First LPS activation	IF lead vs. LPS
1	−50	−40	−40	none	–
2	−20	0	0	none	–
3	−20	−10	−10	06 Jun, 19:42	33.5 h
4	+120	+120	+120	15 Jul, 17:20	4.8 h

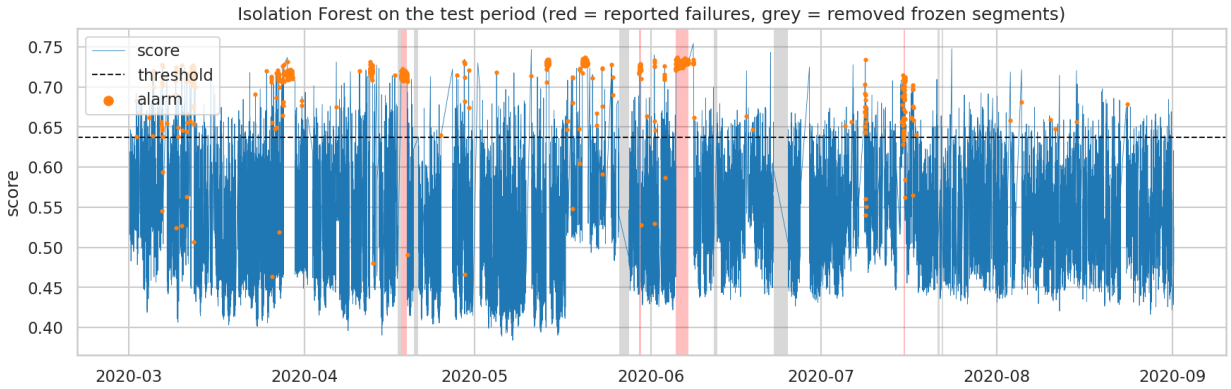


Figure 5: Isolation Forest scores and alarms on the test period. Red: reported failures; grey: removed frozen segments.

Four observations follow. First, the Isolation Forest detects every failure and does so 10 to 20 minutes earlier than the threshold rule for failures 1–3; like the threshold rule, it warns two hours ahead only for failure 4. Second, it produces about twice as many false-alarm episodes (0.52 instead of 0.25 per day), many of them short and scattered over the summer (Figure 5), even though its window-level precision and F1 are higher. Judged only by the window-level F1, the Isolation Forest would appear clearly better; the event-level view shows the cost for maintenance staff. This illustrates why the evaluation protocol reports episodes per day. Third, removing frozen segments does not change any lead time on the real data, but it raises the window precision from 0.290 to 0.421 and removes four false-alarm episodes, mainly from the long frozen segment in late June. Fourth, the low-pressure switch did not activate within the evaluation interval of failures 1 and 2 and activated only about 34 hours after the reported start of failure 3. An evaluation that relies only on the "two hours before LPS" criterion would therefore have no reference point for half of the MetroPT-3 failures, which supports reporting both reference points.

The main cause of the false alarms is the distribution shift between February and the later months (Section 6). Figure 6 shows the mean of each scaled test feature: the minimum compressor pressure (`TP2_min`) lies on average 67 training standard deviations away from its February mean. In February the compressor stops in almost every 10-minute window, so the minimum pressure is close to zero with very little variance; in summer, windows in which the compressor runs under load throughout occur more often. The Isolation Forest is not affected by the scaling itself, because its random splits are drawn between the minimum and maximum of each feature, but the shifted test windows lie outside the February range and are isolated quickly. Any change to the features or the training period to address this will be justified by this analysis of the training data and not selected on the test failures.

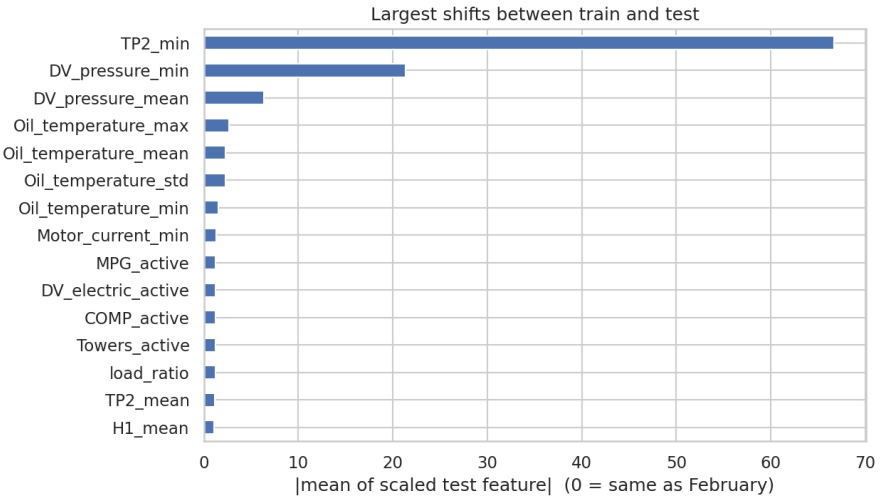


Figure 6: Absolute mean of the scaled test features (0 = same as February). The 15 largest shifts are shown.

6 Assumptions and Challenges

6.1 Assumptions

1. **February is normal.** The training month contains no reported failure and is assumed to represent normal operation. Unreported faults in February would raise the thresholds and reduce sensitivity.
2. **The maintenance reports are correct and complete.** Only the four published failure periods count as failures. Their start times are the reported times, not the physical onset of the leaks; the thesis of Barros [22] even dates failure 4 two days later than the dataset page.
3. **Frozen segments are logging artefacts**, not real machine states, and can be removed.
4. **Ten-minute windows** are short enough to react within the two-hour horizon and long enough to contain several compressor cycles.

6.2 Limitations and Challenges

1. **Four failures.** Every failure changes the event recall by 25 percentage points, so differences between detectors must be interpreted with care and are reported per failure.
2. **Weak early signal.** The leak signature is strong during a failure but weak one day before (Section 2.2); some failures may only be detectable after their reported start.
3. **Distribution shift.** February has a lower duty cycle and oil temperature than the later months. The prototype confirms that a detector trained on February flags parts of normal summer operation (Section 5). For the LSTM autoencoder, features with almost no variance in February, such as `TP2_min`, will also dominate the reconstruction

error after standardisation; robust scaling or removing such features will be examined. We keep February as the only training month to avoid leakage and report the shift as a limitation; it is a change in the input distribution rather than concept drift in the supervised sense [23].

4. **Recording gaps** (17.7 % of the period) shorten the usable data and can hide the onset of a failure.
5. **Threshold sensitivity.** Window length, the IQR factor and the filter constant influence lead time and false alarms. They are fixed in advance here; their sensitivity will be studied in phase 2 without selecting values on the test failures.
6. **Computation.** The LSTM autoencoder requires a GPU for reasonable training times; Google Colab is used.
7. **Explanation cost.** Kernel SHAP requires many model evaluations; explanations are therefore computed only for alarm windows, with a sample of February windows as background.

7 Conclusion

This report defined the theoretical and methodological basis of our case study. Early warning of air leaks is formulated as unsupervised anomaly detection on windows of APU sensor data, motivated by the physical leak signature of a compressor that runs under load almost continuously during a failure. Three detectors of increasing complexity, a threshold rule, an Isolation Forest and an LSTM autoencoder, share one preprocessing pipeline that removes frozen logging segments and respects recording gaps, one alarm logic, and one event-level evaluation with lead time, false alarms per day and lead time relative to the low-pressure signal. The modular architecture separates data, preprocessing, detection, alarm logic and evaluation, which prevents leakage and guarantees that all detectors are judged by the same code. A first prototype shows that the Isolation Forest detects all four failures and reacts earlier than the threshold rule for three of them, but raises about twice as many false alarms, mainly because of the distribution shift between February and the summer months; it also shows that the low-pressure switch is not a usable reference for two of the four failures.

The second methodology phase will implement the LSTM autoencoder and the SHAP explanation module, define the experimental scenarios (detector comparison, frozen-segment ablation and parameter sensitivity) and run the full evaluation.

References

[1] Bruno Veloso, Rita P. Ribeiro, João Gama, and Pedro Mota Pereira. The MetroPT dataset for predictive maintenance. *Scientific Data*, 9:764, 2022. doi: 10.1038/s41597-022-01877-3.

[2] Narjes Davari, Bruno Veloso, Rita Ribeiro, and João Gama. MetroPT-3 dataset. UCI Machine Learning Repository, 2021.

[3] Sebastian Schmidl, Phillip Wenig, and Thorsten Papenbrock. Anomaly detection in time series: A comprehensive evaluation. *Proceedings of the VLDB Endowment*, 15(9):1779–1797, 2022. doi: 10.14778/3538598.3538602.

[4] Siwon Kim, Kukjin Choi, Hyun-Soo Choi, Byunghan Lee, and Sungroh Yoon. Towards a rigorous evaluation of time-series anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, volume 36, pages 7194–7201, 2022. doi: 10.1609/aaai.v36i7.20680.

[5] Narjes Davari, Bruno Veloso, Rita Paula Ribeiro, and João Gama. Detecting and explaining anomalies in the air production unit of a train. In *Proceedings of the 39th ACM/SIGAPP Symposium on Applied Computing*, pages 358–364, 2024. doi: 10.1145/3605098.3635906.

[6] Narjes Davari, Bruno Veloso, Gustavo de Assis Costa, Pedro Mota Pereira, Rita P. Ribeiro, and João Gama. A survey on data-driven predictive maintenance for the railway industry. *Sensors*, 21(17):5739, 2021. doi: 10.3390/s21175739.

[7] Lukas Ruff, Jacob R. Kauffmann, Robert A. Vandermeulen, Grégoire Montavon, Wojciech Samek, Marius Kloft, Thomas G. Dietterich, and Klaus-Robert Müller. A unifying review of deep and shallow anomaly detection. *Proceedings of the IEEE*, 109(5):756–795, 2021. doi: 10.1109/JPROC.2021.3052449.

[8] Fei Tony Liu, Kai Ming Ting, and Zhi-Hua Zhou. Isolation forest. In *2008 Eighth IEEE International Conference on Data Mining*, pages 413–422, 2008. doi: 10.1109/ICDM.2008.17.

[9] Mayu Sakurada and Takehisa Yairi. Anomaly detection using autoencoders with nonlinear dimensionality reduction. In *Proceedings of the MLSDA 2014 2nd Workshop on Machine Learning for Sensory Data Analysis*, pages 4–11, 2014. doi: 10.1145/2689746.2689747.

[10] Sepp Hochreiter and Jürgen Schmidhuber. Long short-term memory. *Neural Computation*, 9 (8):1735–1780, 1997. doi: 10.1162/neco.1997.9.8.1735.

[11] Pankaj Malhotra, Anusha Ramakrishnan, Gaurangi Anand, Lovekesh Vig, Puneet Agarwal, and Gautam Shroff. LSTM-based encoder-decoder for multi-sensor anomaly detection. arXiv preprint arXiv:1607.00148, 2016. URL `https://arxiv.org/abs/1607.00148`.

[12] Miguel E. P. Silva, Bruno Veloso, and João Gama. Predictive maintenance, adversarial autoencoders and explainability. In *Machine Learning and Knowledge Discovery in Databases: Applied Data Science and Demo Track (ECML PKDD 2023)*, Lecture Notes in Computer Science, pages 260–275, 2023. doi: 10.1007/978-3-031-43430-3_16.

[13] João Gama, Rita P. Ribeiro, Saulo Mastelini, Narjes Davari, and Bruno Veloso. A neuro-symbolic explainer for rare events: A case study on predictive maintenance. arXiv preprint arXiv:2404.14455, 2024.

[14] Rita P. Ribeiro, Pedro Pereira, and João Gama. Sequential anomalies: A study in the railway industry. *Machine Learning*, 105(1):127–153, 2016. doi: 10.1007/s10994-016-5584-6.

[15] Kyle Hundman, Valentino Constantinou, Christopher Laporte, Ian Colwell, and Tom Soderstrom. Detecting spacecraft anomalies using LSTMs and nonparametric dynamic thresholding. In *Proceedings of the 24th ACM SIGKDD International Conference on Knowledge Discovery & Data Mining*, pages 387–395, 2018. doi: 10.1145/3219819.3219845.

[16] Alexander Lavin and Subutai Ahmad. Evaluating real-time anomaly detection algorithms – the Numenta anomaly benchmark. In *2015 IEEE 14th International Conference on Machine Learning and Applications (ICMLA)*, pages 38–44, 2015. doi: 10.1109/ICMLA.2015.141.

[17] Nesime Tatbul, Tae Jun Lee, Stan Zdonik, Mejbah Alam, and Justin Gottschlich. Precision and recall for time series. In *Advances in Neural Information Processing Systems 31 (NeurIPS 2018)*, 2018. URL `https://papers.nips.cc/paper/2018/hash/8f468c873a32bb0619eaeb2050ba45d1-Abstract.html`.

[18] Renjie Wu and Eamonn Keogh. Current time series anomaly detection benchmarks are flawed and are creating the illusion of progress. *IEEE Transactions on Knowledge and Data Engineering*, 2021. doi: 10.1109/TKDE.2021.3112126. Early access.

[19] Simon Vollert, Martin Atzmueller, and Andreas Theissler. Interpretable machine learning: A brief survey from the predictive maintenance perspective. In *2021 26th IEEE International Conference on Emerging Technologies and Factory Automation (ETFA)*, pages 1–8, 2021. doi: 10.1109/ETFA45728.2021.9613467.

[20] Scott M. Lundberg and Su-In Lee. A unified approach to interpreting model predictions. In *Advances in Neural Information Processing Systems 30 (NIPS 2017)*, 2017. URL `https://papers.nips.cc/paper/7062-a-unified-approach-to-interpreting-model-predictions`.

[21] Logan Cummins, Alex Sommers, Somayeh Bakhtiari Ramezani, Sudip Mittal, Joseph Jabour, Maria Seale, and Shahram Rahimi. Explainable predictive maintenance: A survey of current methods, challenges and opportunities. arXiv preprint arXiv:2401.07871, 2024. URL `https://arxiv.org/abs/2401.07871`.

[22] Mariana Chaves de Barros. Predictive maintenance for air production unit in EuroTram vehicles. Master's thesis, Faculdade de Economia, Universidade do Porto, 2020. URL `https://repositorio-aberto.up.pt/bitstream/10216/141232/2/433646.pdf`.

[23] João Gama, Indrė Žliobaitė, Albert Bifet, Mykola Pechenizkiy, and Abdelhamid Bouchachia. A survey on concept drift adaptation. *ACM Computing Surveys*, 46(4):1–37, 2014. doi: 10.1145/2523813.